



2



20



3000 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

AVL + Heap

ملحق



RB Informatics; 25/5/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

المحاضرة الرابعة "AVL Tree" نظري :

أولاً : مشكلة binary search tree : الارتفاع غير قابل للتحكم ويعتمد كلياً على كيفية ادخال قيم ال keys لل tree وهذا هو عيب BST لنحل هي المشكلة أجتت فكرة ال AVL Tree

■ AVL Tree :

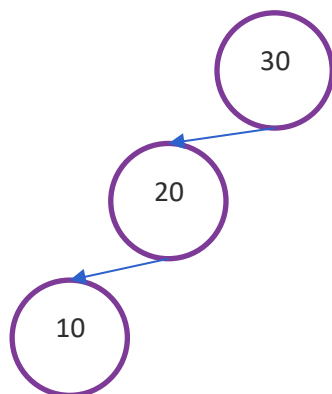
يجب أن تكون شجرة بحث ثنائية BST لكل عقدة في الشجرة، يجب أن تكون قيمة معامل التوازن حصراً تنتمي للمجموعة $\{-1, 0, 1\}$

Balance factor = height of left subtree - height of right subtree

$$|bf| = |h_l - h_r| \leq 1$$

Left Case:

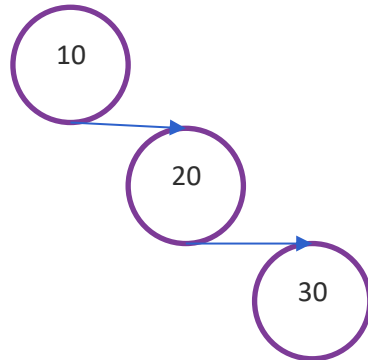
- Occurs when a node is inserted into the left subtree of the left child, causing the balance factor to become **more than +1**.
- Fix:** Perform a single **right** rotation.



العقدة 30 معامل توازنها 2
لا ينتمي لـ $\{-1, 0, 1\}$

Right-Right Case:

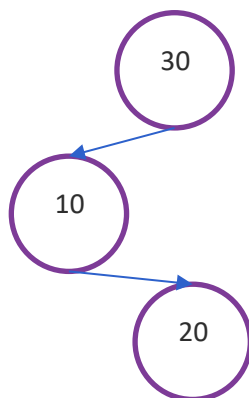
- Occurs when a node is inserted into the right subtree of the right child, making the balance factor **less than -1**.
- Fix:** Perform a single **left** rotation.



العقدة 30 معامل توازنها -2
لا ينتمي لـ $\{-1, 0, 1\}$

Left-Right Case:

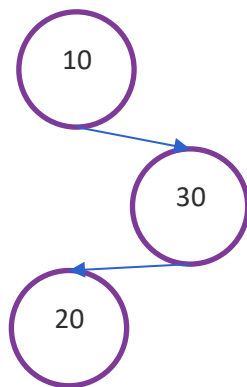
- Occurs when** a node is inserted into the right subtree of the left child, which disturbs the balance factor of an ancestor node, making it **left-heavy**.
- Fix:** Perform a left rotation on the left child, followed by a right rotation on the node.



العقدة 30 معامل توازنها 2

Right-Left Case:

- Occurs when a node is inserted into the left subtree of the right child, which disturbs the balance factor of an ancestor node, making it **right-heavy**.
- Fix:** Perform a right rotation on the right child, followed by a left rotation on the node.



العقدة 10 معامل توازنها -2



■ الأسئلة:

1. عند القيام بعملية حذف (Deletion) من شجرة AVL ، ما هو أقصى عدد من عمليات الدوران (Rotations) التي قد نحتاجها لإعادة موازنة الشجرة على طول المسار حتى الجذر؟

A. صفر عمليات دائماً

B. $O(n)$

C. $O(\log n)$

D. دوران واحد فقط $O(1)$

الإجابة صحيحة: C

التعليق: في عملية الحذف، إعادة التوازن عند عقدة قد يؤدي إلى تقليص ارتفاع الفرع، مما ينقل الاختلال للأعلى ويتطلب دورانات إضافية على طول المسار.

2. في شجرة AVL، ما هو ارتفاع العقدة الورقية (Leaf Node) التي ليس لها أبناء؟

A. 1

B. يساوي عدد العقد الكلي في الشجرة

C. -1

D. 0

الإجابة صحيحة: D

العقدة التي لا تملك أي أبناء (ورقة) يكون ارتفاع فرعها الأيسر والأيمن منعدماً، وبالتالي ارتفاعها يساوي صفر.

3. إذا كان عامل التوازن لعقدة ما يساوي $+2$ ، فهذا يعني أن الشجرة؟

- A. غير متوازنة والفرع الأيسر أطول من الأيمن بدرجتين
- B. غير متوازنة والفرع الأيمن أطول من الأيسر بدرجتين
- C. فارغة تماماً ولا تحتوي على عقد
- D. متوازنة تماماً ولا تحتاج أي تعديل

الإجابة صحيحة: A

بما أن الناتج موجباً $(+2)$ ، فهذا يدل حسب قانون الطرح (الأيسر - الأيمن) على أن الطرف الأيسر هو الأثقل والأطول.

4. تخيل أنك تقوم بكتابة كود لعقدة شجرة AVL، وقمت بحساب عامل التوازن لعقدة ما فكان $+2$ ، وعند فحص ابنها الأيسر تبين أن عامل توازنه يساوي 0. هذه الحالة الخاصة (التي تحدث في الحذف فقط)، ما هو الدوران الذكي والفعال لحلها؟

- A. تخطي العقدة دون دوران لأن توازن الابن سليم ويساوي صفر
- B. دوران أيسر مفرد (Single Left Rotation)
- C. دوران أيمن مفرد (Single Right Rotation)
- D. دوران أيسر-أيمن مزدوج (Left-Right Rotation)

الإجابة صحيحة: C

بما أن توازن الابن صفر (لا يوجد تعرج متعاكس)، فإن الدوران المفرد الأيمن كافٍ تماماً لإعادة التوازن وضمان بقاء معاملات التوازن بين -1 و 1 .

5. ثناء إجراء دوران يعني (Right Rotation) على عقدة الأصل b والتي ابنها الأيسر هو a، ماذا يحدث للابن اليميني للعقدة a؟

- A. يصبح ابناً يمينياً لعقدة الأصل b
- B. يصبح ابناً يسارياً لعقدة الأصل b
- C. يُحذف تلقائياً من الشجرة
- D. يتحول ليكون جذراً جديداً للشجرة كاملة

الإجابة صحيحة: B لأن العقدة b تهبط لتصبح الابن الأيمن للعقدة a، وبالتالي تأخذ العقدة b الابن الأيمن القديم لـ a وتجعله ابناً يسارياً لها للحفاظ على خصائص شجرة البحث الثنائية (BST)

6. إذا قمنا بفحص عقدة الخلل فوجدنا معامل توازنها يساوي -2، بينما معامل توازن ابنها اليميني يساوي +1، فما هو الدوران المناسب لإعادة التوازن للجزء المعطوب؟

A. دوران يساري متبوع بدوران يميني (LR Rotation)

B. دوران يميني يساري (Right-Left Rotation)

C. دوران يساري منفرد (Left Rotation)

D. دوران يميني منفرد (Right Rotation)

الإجابة صحيحة: B

7. خلال عملية الصعود لفحص التوازن بعد الإضافة، ماذا نفعل عند العثور على أول عقدة غير متوازنة؟

A. نصلحها بالدوران المناسب ثم نستمر بفحص الشجرة حتى الجذر خوفاً من وجود اختلالات أخرى

B. نقوم بحذف العقدة المضافة للتو لإلغاء المشكلة

C. نطبق الدوران المناسب عليها فوراً ونقوم بالتوقف عن الصعود لأن الشجرة عادت بالكامل للتوازن

D. نقوم بنقل عقدة الجذر لتكون أسفل أول عقدة غير متوازنة

الإجابة صحيحة: C

■ التعليل:

■ عند الإضافة (Insertion) بمجرد العثور على أول عقدة غير متوازنة والدوران حولها، نتوقف فوراً ، لأن الدوران يعيد ارتفاع الشجرة الفرعية إلى ما كان عليه قبل الإضافة، فلا يتأثر أي أب في الأعلى.

■ عند الحذف (Deletion) هنا العكس تماماً، لو كان السؤال عن الحذف لأن الدوران عند عقدة معينة لتعويض النقص قد يقلل من ارتفاع الشجرة الفرعية بالكامل، مما قد يسبب خللاً في العقد الأعلى، لذا نضطر لمتابعة الصعود والفحص حتى نصل إلى الجذر.

8. لدى إضافة العدد (12) إلى شجرة AVL تحوي العقد {20 (جذر)، 10، 30، 7، 17، 26، 15}. ما هي العقدة التي يحصل عندها أول خلل (امتداد غير متوازن) أثناء الصعود؟ وما هو الدوران المناسب لعلاجها؟

A. العقدة 20 ويتم إصلاحها بدوران يساري

B. العقدة 10 والحل هو دوران يميني يساري

C. العقدة 17 ويحدث عندها خلل (القيم غير متوازنة وبحسب شكل المسار نطبق دوراناً يمينياً على العقدة 17)

D. العقدة 15 والحل هو دوران يساري يميني

9. كيف نختار مسار الإصلاح وال Subtree ذات الارتفاع الأطول لعقدة الخلل في حالتي الإضافة والحذف؟

- A. في الإضافة نختار عشوائياً وفي الحذف نركز على الفرع الأقصر دائماً لتعويضه
- B. في الإضافة: عقدة الخلل + ابنها + حفيدها. في الحذف: نأخذ عقدة الخلل مع فرع الابن الذي يحتوي الارتفاع الأكبر (نركز على الفرع الأكثر ثقلًا)
- C. نعتمد دوماً على العقد اليمنى بغض النظر عن الارتفاعات
- D. نأخذ العقدة المفقودة وننشئ لها شجرة فرعية منفصلة

B الإجابة صحيحة:

قاعدة مهمة جداً:

- في حالة الإضافة (Insertion): نبحث عن المسار المسبب للمشكلة صعوداً من العقدة المضافة حديثاً، فنحدد عقدة الخلل (الجد)، ثم ابنها على مسار الإضافة، ثم حفيدها على نفس المسار لنعرف نوع الدوران (LL, RR, LR, RL)
- في حالة الحذف (Deletion): الخلل يحدث لأن هناك فرعاً أصبح خفيفاً (نقص ارتفاعه)، ولكي نحدد الدوران المناسب لإعادة التوازن، ننظر إلى الفرع الآخر الأثقل (الابن صاحب الارتفاع الأكبر) لإعادة توزيع العقد منه وتعديل الكفة.

10. ما هو التعقيد الزمني الأقرب (Tightest upper bound) لعمليات الدوران اليميني واليساري (Right-Rotate و Left-Rotate) المستخدمة في أشجار البحث الثنائية متوازنة الصعود؟

A. $O(1)$

B. $O(\log n)$

C. $O(\log(\log n))$

D. $O(n)$

الإجابة صحيحة: A لأن عملية الدوران لا تقوم بنقل العقد أو المرور عليها واحدة تلو الأخرى؛ كل ما تفعله هو تغيير اتجاه 4 أو 5 مؤشرات (Pointers) فقط في الكود.

لا تقبل بهدف أقل من القمة
فانت نسطيع ذلك.. ونسحق ذلك 🙄❤🔥

11. أي من الحالات الإشارية التالية لمعاملات توازن عقدة الخلل (Z) وولدها الأيمن تؤكد بلا شك أننا أمام حالة (Right-Left) والتي تستدعي دوران 'يميني يساري' مزدوج (Right-Left Rotation) ؟

- A. معامل توازن عقدة الخلل Z هو -2، ومعامل توازن ابنها الأيمن هو +1.
- B. معامل توازن عقدة الخلل Z هو +2، ومعامل توازن ابنها الأيسر هو -1.
- C. معامل توازن عقدة الخلل Z هو -2، ومعامل توازن ابنها الأيمن هو -1.
- D. معامل توازن عقدة الخلل Z هو +2، ومعامل توازن ابنها الأيسر هو 0.

الإجابة صحيحة: A

12. ما الذي يحدث بدقة لخوارزمية توازن شجرة AVL إذا قمنا بحذف العقدة الوحيدة التي تمثل الجذر الرئيسي لشجرة غاية في الضخامة والعمق ؟

- A. نستبدل الجذر بالـ Successor (الأصغر بالفرع الأيمن) أو الـ Predecessor (الأكبر بالأيسر) تماشياً مع BST ، ثم نقوم بالمرور التراجعي من موقع الاستبدال الأصلي للأعلى صعوداً للجذر مدققين توازن معاملات الارتفاع مع استدعاء الدورانات حينما وُجد الخلل.
- B. تتوقف الشجرة بالكامل ويتعطل تصفحها لعدم وجود جذر آخر بديل.
- C. يصعد الابن الأيسر للجذر فوراً ليمسك قمة الشجرة بدون فحص أو تعديل لباقي الفروع.
- D. يتم حذف كافة العقد تلقائياً بالاعتماد على مكنة الـ Garbage Collector بالمعالج.

الإجابة صحيحة: A

13. إذا كان لدينا شجرة AVL ضخمة تحتوي على 1,000,000 عقدة مفرزة، فما هو أقصى ارتفاع نظري ممكن لهذه الشجرة في أسوأ الحالات السيئة، مقارنة بشجرة البحث الثنائية العادية (BST) ؟

- A. لا يمكن نظرياً أن يتجاوز الارتفاع حوالي 28 طابقاً وفقاً لقيد $1.44 \log(n+2)$ ، بينما في BST العادية قد ينهار الارتفاع لـ 1,000,000 طابق.
- B. سيرتفع ليتطابق مع الـ BST عند 500,000 طابق تقريباً.
- C. يتوقف الارتفاع تماماً عند 10 طوابق نظراً لخاصية الدوران المحمية.
- D. الارتفاع سيكون صفراً لسرعة عمليات الذاكرة العشوائية.

الإجابة صحيحة: A

فائدة :

■ شجرة الـ AVL تضمن دائماً بقاء الشجرة متوازنة، وأقصى ارتفاع لها في أسوأ الحالات (Worst-case) لا يتعدى تقريباً $1.44 \log(n+2)$ بالنسبة لمليون عقدة، الناتج سيكون حوالي 28 مستواً (أو طباقاً). أما شجرة البحث العادية BST، فإذا أضفنا العناصر وهي مرتبة (مفرزة)، ستتحوّل الشجرة إلى ما يشبه القائمة المتصلة (Skewed Tree) وينهار ارتفاعها ليصل لمليون طباق كاملة، وهو ما يجعل الـ AVL متفوقة جداً هنا.

14. شجرة AVL متوازنة تحوي العقد بالبنية والمسارات {50, 40, 60, 30}. قمنا بحذف العقدة (60). تتبع العقد صعوداً للقمّة، واكشف أين يظهر أول خلل بعد حذفها وما هو العلاج الفوري لإعادة التشكيل؟

- A. يحدث أول خلل عدم توازن عند العقدة 50 (معامل التوازن يساراً أصبح 2 بينما يميناً 0)، ونصلحه فوراً بدوران يميني (Right Rotation) حول العقدة 50 لتصبح 40 هي الجذر لـ 30 و 50.
- B. يحدث الخلل عند العقدة 30 والحل هو دوران مزدوج RL للأب 40.
- C. تبقى الشجرة متوازنة تلقائياً ولا يحدث أي تغيير بالمعامل لخلل الـ 60.
- D. يحدث الخلل عند العقدة 40 وتطرد الـ 30 من الشجرة تلقائياً.

الإجابة صحيحة: A

15. ما هو التعقيد الزمني في أسوأ الحالات (Worst-case Time Complexity) لعمليتي الإضافة (Insertion) و الحذف (Deletion) في شجرة AVL تحتوي على n من العقد؟

- A. $O(1)$
- B. $O(\log n)$
- C. الإضافة تأخذ $O(\log n)$ بينما الحذف يأخذ $O(n)$
- D. $O(n)$

الإجابة صحيحة: B

(الإضافة والحذف) كلاهما لهما تعقيد زمني في أسوأ الحالات هو $O(\log n)$ لأن طول مسار البحث أو إعادة التوازن في شجرة AVL لا يتجاوز ارتفاع الشجرة (الذي هو لوغاريتمي).

16. ما هو أقصى ارتفاع نظري ممكن لأي شجرة AVL تحتوي على 7 عقد؟ (افتراض أن ارتفاع الشجرة التي تحتوي على عقدة واحدة فقط هو 0).



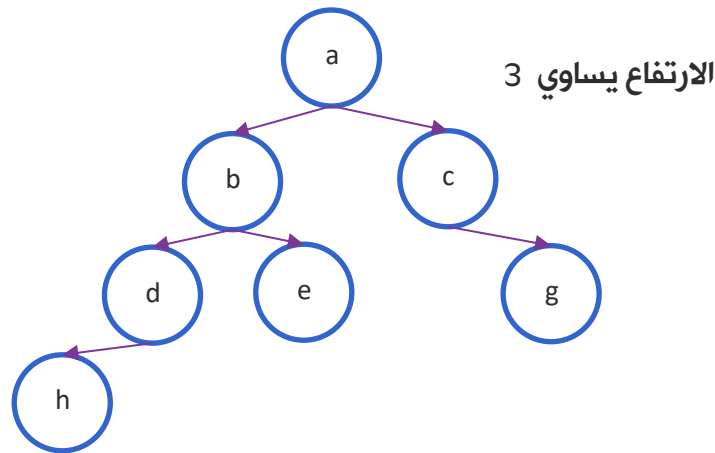
A. 2

B. 3

C. 4

D. 5

الإجابة صحيحة: B



"AVL Tree" عملي

17. عند حدوث خلل توازن من النوع الأيسر-الأيمن (LR) Left-Right عند عقدة معينة، ما هي التدويرات المتتالية المطبقة برمجياً لاستعادة التوازن؟

- A. دوران يساري (leftRotate) على العقدة نفسها ثم يميني (rightRotate)
- B. دوران يساري (leftRotate) على الابن الأيسر، ثم دوران يميني (rightRotate) على العقدة نفسها
- C. دوران يميني (rightRotate) على الابن الأيمن، ثم دوران يساري (leftRotate) على العقدة نفسها
- D. دورتان يمينيتان متتاليتان على العقدة والابن

الإجابة صحيحة: B

18. في خوارزمية الدوران اليميني البرمجية (rightRotate(NODE y) ، لماذا يتم تحديث وحساب ارتفاع العقدة الفرعية الهابطة (y) قبل حساب ارتفاع العقدة الصاعدة الجديدة (x) ؟

- A. لأن y أصبحت ابناً لـ x فارتفاعها المحدث ضروري لحساب ارتفاع x الرئيسي الجديد
- B. لأن x أكبر أبجدياً من y وبالتالي يتم حسابها ثانياً بقواعد BST
- C. هذا غير صحيح، الكود يحسب ارتفاع x أولاً في كل السطور
- D. لتخفيض التعقيد الإجمالي وإعادة ضبط التوازن تلقائياً دون استدعاء عودي

الإجابة صحيحة: A

18. إذا كانت العقدة ممررة كقيمة فارغة (null) إلى التابع التالي:

```
int height(Node node)
{ if (node == null) return ...; return node.height; }
```

ما هي القيمة التي يجب إرجاعها مكان النقاط لتجنب تعطل الكود ولضمان حساب الارتفاعات بشكل سليم تالياً؟

- A. إرجاع القيمة 0 مباشرة
- B. إرجاع القيمة -1
- C. إرجاع القيمة 1
- D. توليد استثناء NullPointerException

الإجابة صحيحة: B

19. بفرض أننا نقوم بإجراء دوران يميني (Right Rotate) حول العقدة (y) ، كود التحديث يبدو كالتالي :

```
rightRotate(Node y) {
    Node x = y.left; Node T2 = x.right; x.right = y; y.left = T2;
    ... تحديث الارتفاعات }
```

ما هو الترتيب الصحيح لتحديث ارتفاعات العقد لضمان سلامة العمليات اللاحقة؟

- A. تحديث ارتفاع x أولاً ثم ارتفاع y
- B. تحديث ارتفاع y أولاً ثم ارتفاع x
- C. تحديث الارتفاعين معاً في نفس السطر بلغة البرمجة
- D. لا حاجة لتحديث ارتفاعاتهم، بل يتم تحديث ارتفاع الجذر الأب فقط

الإجابة صحيحة: B

20. عند حذف العقدة (d) التي تمتلك ابنين مستقلين في شجرة AVL برمجياً، كيف يتم استبدال قيمتها لمواصلة شروط أشجار البحث الثنائية (BST) ؟

- A. نأخذ أصغر قيمة في فرعها الأيمن (Inorder Successor) باستدعاء minValueNode
- B. نأخذ عشوائياً كائن العقدة الأقرب إليها برمجياً
- C. نحذف الفرع الأيسر بالكامل لإفساح المجال لليمين
- D. نستبدلها دائماً بأعلى ورقة من الشق الأيسر لضمان استقرار AVL

الإجابة صحيحة: A

21. بفرض أن لدينا شجرة AVL وجذرها له معامل توازن يساوي 1. عند حذف عقدة من الفرع الأيمن، ما هو معامل التوازن المحتمل الجديد للجذر، وهل سيستدعي ذلك دوراناً إصلاحياً؟

- A. سيصبح معامل التوازن 0، ولن نحتاج إلى تدوير
- B. سيصبح معامل التوازن 2، وسنحتاج قطعاً إلى تدوير لاستعادة الهيكل المتزن
- C. سيصبح معامل التوازن -1، وسنقوم بتدوير يساري مزدوج
- D. ستصبح قيمة التوازن غير معرفة وتطلق دالة insert خطأ

الإجابة صحيحة: B

22. تتبع برمجياً عودة الارتفاعات التراكمية في مكس النداءات (Call Stack) للتابع insert: إذا قمنا بإدراج عقدة في شجرة AVL ذات عمق كبير، ما هي الآلية العملية المتبعة لتحديث قيمة ارتفاع العقد (Variables update)؟

- A. $O(N^2)$ يتم استدعاء دالة مسافات مستقلة من الجذر للبحث عن العقدة المضافة وإعادة كتابة ارتفاعها
- B. مع تراجع مكس النظام العودي (Recursive Backtracking)، يتم تحديث ارتفاع كل عقدة مرت بها دالة insert فوراً بالصيغة: $height = 1 + \text{Math.max}(height(\text{left}), height(\text{right}))$
- C. الارتفاع يزداد ذاتياً بمقدار 1 في كل العقد بشكل معولج خارجياً دون استدعاء معادلة max.
- D. يتم فرض قيمة ارتفاع ثابتة لكل عقد المكس مساوية للارتفاع الأقصى للشجرة.

الإجابة صحيحة: B

في السطور البرمجية المخصصة لدالة الحذف، تجد هذا الشرط المفطلي :

(if (balance > 1 && getBalance(root.left) >= 0) return rightRotate(root));

ما الذي يثبتته فحص $\text{getBalance}(\text{root.left}) \geq 0$ بدقة متناهية مقارنة بـ $\text{getBalance}(\text{root.left}) > 0$ ؟

A. التحقق من علامة المساواة ($= 0$) في الحذف هو شرط أساسي للتعامل مع الحالة التي يكون فيها فرعا الابن الأيسر متساويين بالارتفاع، مما يسمح بحسم التوازن بدوران يميني واحد بدلاً من اللجوء بالخطأ لتراكم تدويرات مزدوجة

B. المساواة تمنع حدوث انخفاض مستديم بالذاكرة للأطراف الصغرى.

C. لا يوجد أي فرق منطقي، ومجرد زيادة للكتابة البرمجية.

D. الشرط يسرع تفتيش الأوراق الميتة لإزالتها من الجداول.

الإجابة صحيحة: A

23. عندما يكون معامل التوازن أكبر من 1 والقيمة المدخلة أصغر من مفتاح الابن الأيسر:

(balance > 1 && key < node.left.key)

ما هو الترتيب البرمجي الصحيح للتدويرات المطبقة لاستعادة توازن الشجرة؟

A. دوران يساري (leftRotate) على الابن الأيسر أولاً، ثم إرجاع نتيجة الدوران اليميني (rightRotate) للعقدة الحالية

B. دوران يميني على العقدة الحالية أولاً، ثم دوران يساري على الابن الأيسر

C. دوران يساري على العقدة الحالية مباشرة دون تعديل الأبناء

D. دوران يميني على العقدة الحالية فقط

الإجابة صحيحة: D

بالاستناد إلى الصورة المرفقة من كود المحاضرة، قارن بعين المطور بين شروط الحالتين المزدوجتين للالتواء الداخلي

(السطرين 5 و 10): في الـ LR: if (balance > 1 && key > node.left.key) وفي الـ RL :

if (balance < -1 && key < node.right.key)

كيف تساهم المقارنة لـ key مع مفتاح ابن العقدة الكودية في التفريق الدقيق بين التدوير المفرد والمزدوج؟

A. تحدد هذه المقارنات ما إذا كانت العقدة الجديدة المضافة قد سلكت مساراً 'داخلياً' (أي اليمين للابن الأيسر، أو

اليسار للابن الأيمن)، مما يستدعي التدوير المزدوج لتصحيح الالتواء الداخلي وتحويله إلى تباعد خارجي متسق

قبل التدوير النهائي للأب

- B. يقوم النظام بمطابقة قيم المفاتيح لمنع التكرار فقط دون التأثير على حسابات التوازن الجانبي.
- C. هذه الشروط وُضعت للتحري عن القيم الصفرية لتفادي أخطاء الـ Pointer Exception.
- D. تضمن هذه الشروط مطابقة الأوزان لكي لا يتجاوز ميزان الارتفاع 3.

الإجابة الصحيحة: A

المحاضرة الثالثة نظري heap

1. عند مقارنة استخدام الكومة (Heap) مقابل المصفوفة المرتبة (Sorted Array) لتنفيذ رتل أولويات للمهام، أي العبارات التالية تصف الفروقات الزمنية وجدارة الاستخدام بشكل دقيق؟

- A. المصفوفة المرتبة أفضل لإجراء العمليات لأن استخراج العنصر المستهدف بالكامل يتم في زمن ثابت ومباشر $O(1)$
- B. الكومة أفضل بكثير لأن تكلفة الإدخال (Enqueue) فيها هي $O(\log N)$ والاستخراج (Dequeue) هو $O(\log N)$ ، في حين أن الإدخال في المصفوفة المرتبة يكلف $O(N)$ للقيام بعمليات إزاحة العناصر بالذاكرة لحفظ الترتيب الكامل.
- C. تتطلب الكومة استخدام خلايا ربط، عملاقة ومصفوفات ثانوية تزيد من كلفة التعقيد المساحي لمرتبة $O(N^2)$ مقارنة بالمصفوفة البسيطة.
- D. لا توجد فروق جوهرية بين الهيكلين ويتم اختيار المصفوفة المرتبة دوماً لسلامة استيعابها وفهمها برمجياً.

الإجابة الصحيحة: B

2. عند جريان عملية Dequeue لحذف العضو الأهم في رتل أولويات مبني على Max-Heap، نقوم بمبادلة الجذر بأخر ورقة وحذفها، ثم تطبيق Max_Heapify على الجذر المستبد. ما الذي يضمن علمياً أن استدعاء Heapify الفردي هذا كافٍ لإرجاع الكومة لواقعها السليم الصارم؟

- A. أن كلاً من الشجرتين الفرعيتين (اليسرى واليمنى) للجذر الأصلي لم يتم مساسهما بالأصل وبقيتا كومتين قصويين صحيحتين ومثليتين بنسبة 100٪، فالخلل بالكامل ينحصر في قيمة الجذر الجديد المستقطب، وصلاحيات Heapify تبحث عن هذا الخلل وتوجه الدفع المناسب للأسفل في مسلك وحيد مساو لعمق الشجرة.
- B. لأن القيمة القادمة من نهاية الشجرة هي بالضرورة أصغر قيمة موجودة، فلا يمكن منطقياً أن تخلق تضارباً مع طبقات المستويات اللاحقة.

- C. لأن الأجهزة الحديثة تستشعر عمليات الإلغاء وتقوم بإزاحة المستويات تلقائياً لاستقرار الحجم.
- D. لأن تقلص حجم الكومة يلغي تلقائياً الطبقة السفلية التي وقع فيها الاختلال دون الحاجة لفحص دقيق للآباء والمستويات.

الإجابة الصحيحة: A

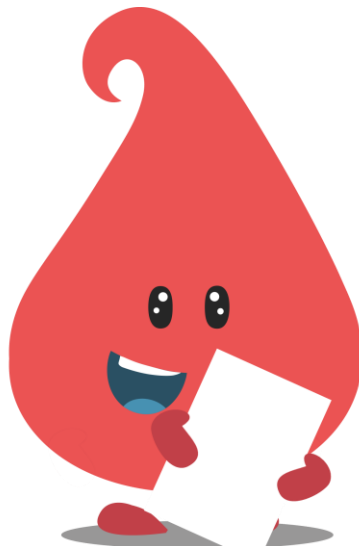
3. إذا كنا نمثل شجرة ثنائية مكتملة (Complete Binary Tree) في مصفوفة أحادية تبدأ من الفهرس 1. إذا كانت العقدة X تقع عند الفهرس 11 في مصفوفة مكتملة تماماً، فما هو الفهرس الذي تقع فيه العقدة الأب (Parent) لهذه العقدة، وفي أي مستوى (Level) تقع العقدة X إذا كان الجذر يمثل المستوى 0؟

- A. الأب عند الفهرس 5، والمستوى هو 3.
- B. الأب عند الفهرس 5، والمستوى هو 4.
- C. الأب عند الفهرس 6، والمستوى هو 3.
- D. الأب عند الفهرس 6، والمستوى هو 4.

الإجابة الصحيحة: A

4. في بنية الكومة القصوى (Max-Heap)، ما الذي يمكننا استنتاجه بالتأكيد حول العلاقة بين قيمتي عقدتين شقيقتين (Siblings) تشتركان في نفس الأب؟

- A. الابن الأيمن يجب دائماً أن يكون أصغر من أو يساوي الابن الأيسر.
- B. الابن الأيسر يجب دائماً أن يكون أصغر من أو يساوي الابن الأيمن.
- C. لا توجد أي علاقة شرطية محددة بين قيم الإخوة؛ فالترتيب بينهما غير مهم لتحقيق شرط الكومة.
- D. يجب أن تكون القيمة في الفهرس الزوجي ضعف القيمة في الفهرس الفردي دائماً.

الإجابة الصحيحة: C

حَدَّثْ لِنَعْسَكَ قُلُوبًا : يَا أَنَا
 وَهَلْ خَلَقَ الْغُورُ إِلَّا لَنَا 🧡😊
 لَنَا بَيْنَ رُكْبِ النَّجَاحِ مَكَانٌ
 وَلَنْ يَمُضِيَ الرُّكْبُ إِلَّا بِنَا 🙏😊



5. إذا كان لديك كومة صغرى (Min-Heap) تحتوي على N من العناصر الفريدة والمخزنة في مصفوفة أحادية تبدأ من الفهرس 1. أين يمكن أن نجد العقدة التي تحمل أكبر قيمة (Maximum) في هذه الكومة، وما هو التعقيد الزمني للوصول إليها؟

- A. يمكن العثور عليها دائماً عند آخر فهرس في المصفوفة $A[N]$ بتكلفة زمنية $O(1)$.
- B. يمكن العثور عليها في النصف الثاني من المصفوفة (عقد الأوراق Leaves) وتحديدًا ضمن الفهارس من $\text{floor}(N/2) + 1$ إلى N بتكلفة زمنية قدرها $O(N)$.
- C. يمكن العثور عليها عند الجذر $A[1]$ بتكلفة زمنية $O(1)$.
- D. لا يمكن العثور عليها أبداً إلا بإعادة بناء الكومة كـ Max-Heap بتكلفة زمنية $O(N \log N)$.

الإجابة الصحيحة: B

6. لدينا مصفوفة أحادية مرتبة تماماً تصاعدياً (Ascending Sorted Array) مثل $[3, 7, 10, 12, 15, 20]$. هل تمثل هذه المصفوفة بنية كومة (Heap) صحيحة دون تعديل عليها؟ وإذا كانت كذلك، فما هو نوعها؟

- A. نعم، تمثل كومة قصوى (Max-Heap) صحيحة حتماً لأنها مرتبة.
- B. نعم، تمثل كومة صغرى (Min-Heap) صحيحة حتماً.
- C. لا، المصفوفة المرتبة تماماً لا يمكن أن تحقق شرط الكومة أبداً بسبب وجود فراغات وهمية.
- D. لا، لأن الكومة تشترط عدم ترتيب عناصر المستويات كاملاً بل جزئياً.

الإجابة الصحيحة: B

7. نفترض أنه تم استدعاء دالة $\text{Max_Heapify}(A, i)$ على الكومة الممثلة بالمصفوفة التالية: $A = [10, 30]$, $[25, 12, 14, 20, 22]$ والفهرس الممرر هو $i=2$ (أي العقدة التي قيمتها 30 بالتمثيل المبتدئ بـ 1). أي من العبارات التالية تصف بشكل دقيق ما سيحدث؟

- A. سيتم إطلاق استدعاء عودي لإصلاح الشجرة الفرعية لأن القيمة 30 أصغر من ابنها.
- B. لن يحدث أي تغيير أو تغيير في المصفوفة لأن الشجرة الفرعية التي جذرها 30 تحقق بالفعل شرط الكومة القصوى (Max-Heap).
- C. سيحدث خلل لأن الشجرة الفرعية اليسرى واليمنى للـ 30 لا تحققان شرط الـ Max-Heap.
- D. ستبدل القيمة 30 مع الجذر 10 مباشرة لأن الـ 30 أكبر من الأب.

الإجابة الصحيحة: B

8. في الكود الوارد لبناء كومة قصوى `build_Max_Heap` (صفحة 10): تبدأ الحلقة `for` من الفهرس `A.length/2` وتتناقص نزولاً إلى 1. لماذا نبدأ من المنتصف بالضبط بدلاً من نهاية المصفوفة؟

- A. لأن النصف الثاني من المصفوفة (من منتصفها لنهايتها) يمثل عقد الأوراق (Leaves) التي لا تملك أبناء أصلاً، وخوارزمية `Max_Heapify` تفترض وجود أبناء لإصلاح العلاقة معهم، لذلك تطبيقها على الأوراق عديم الفائدة وبدؤها من أول عقدة غير ورقة يحمي الأداء والمنطق.
- B. لتوفير مساحة الذاكرة فقط، بينما البدء من النهاية كان سيعطي تعقيداً زمنياً أفضل بكثير.
- C. لأن مستويات الشجرة العليا يجب تقييمها قبل المستويات السفلى لضمان تماسك العلاقات الرأسية.
- D. لأن الأوراق في الكومة القصوى تكون دائماً مرتبة مسبقاً بقيم أصغر من الجذر فلا داعي لاختبارها.

الإجابة الصحيحة: A

9. يعتقد الكثير للوهلة الأولى أن تعقيد بناء الكومة `Build_Max_Heap` هو $O(N \log N)$ بسبب استدعاء دالة `Max_Heapify` التي تكلف $O(\log N)$ لـ $N/2$ من المرات. ولكن التعقيد الحقيقي هو $O(N)$. ما التفسير العلمي والرياضي لهذا السلوك؟

- A. لأن معظم العقد التي نطبق عليها `Heapify` تقع في المستويات السفلية القريبة من الأوراق بامتياز، حيث يكون لارتفاع العقدة قيمة صغيرة جداً (كالارتفاع 1 أو 2)، مما يقلل خطوات الهبوط لأدنى حد. وكلما صعدنا لأعلى الشجرة (حيث الارتفاع $\log N$)، يقل عدد العقد بشكل كبير، مما يجعل مجموع كلف الحركة لجميع العقد محاطاً بـ $O(N)$.
- B. لأن المترجم يقوم بتحسين الاستدعاءات التكرارية وتحويلها للعمل المتوازي للحد من حجم المستويات المستعرضة.
- C. لأن الشجرة مكتملة الارتفاع، مما يمكن من استبدال العناصر وتحريكها بشكل مباشر وعشوائي في زمن ثابت $O(1)$.
- D. بسبب تمرير نصف مصفوفة البيانات فقط إلى الدالة البانية، مما يقلص الحجم ليكون مساوياً لـ $O(N/2)$ والذي يؤول لثابت $O(1)$.

الإجابة الصحيحة: A

10. إذا أردنا استخدام خوارزمية فرز الكومة (Heap Sort) للحصول على مصفوفة مرتبة ترتيباً تصاعدياً (Ascending Order) بالكامل بشكل اقتصادي في الموقع (In-place)، فما المنهجية المتبعة هندسياً؟

- A. نبني أولاً كومة صغرى (Min-Heap)، ثم في كل خطوة نستخرج العنصر الأصغر ونضعه في نهاية المصفوفة.
- B. نبني أولاً كومة قصوى (Max-Heap)، وفي كل خطوة نقوم بتبديل الجذر (الأكبر) مع آخر عنصر في الكومة النشطة، ثم نقلل حجم الكومة بمقدار 1 ونعيد تطبيق Heapify على الجذر الجديد لإصلاح الخل.
- C. نبني كومة قصوى (Max-Heap)، ثم نقوم بعكس المصفوفة بأكملها من البداية للنهاية بتكلفة زمنية قدرها $O(N^2)$.
- D. لا يمكن فرز المصفوفة تصاعدياً مباشرة باستخدام الكومة القصوى بل يجب استخدام الكومة الصغرى حتماً لتشجيع الصعود التصاعدي.

الإجابة الصحيحة: B

11. إذا رغبت في البحث عن عنصر معين (مثلاً الرقم 17) في كومة قصوى (Max-Heap) ممثلة في مصفوفة مسطحة تحتوي N من العناصر الفريدة، أي العبارات تصف بشكل سليم التقييم الزمني والعملي للوصول؟

- A. القيام بالبحث الثنائي (Binary Search) بكفاءة كاملة $O(\log N)$ لكون الشجرة ثنائية متوازنة بالدقة.
- B. يتطلب الأمر إجراء بحث خطي شامل (Linear Search) على المصفوفة لتكون الكلفة $O(N)$ في الحالة الأسوأ، نظراً لأن الكومة تقيد الآباء والأبناء ولكنها لا تضمن أي ترتيب نسبي يميناً أو يساراً لفرز الاتجاهات.
- C. يسهل المرور وتوجيه القفزة دائماً بمقارنة الرقم مع الجذر والنزول للجانب الأكبر بتكلفة مثالية $O(\log N)$.
- D. العملية مستحيلة كلياً ولن نسترد العنصر إلا باستخراج جميع العناصر من الكومة وحفظها في بنية بيانات مساعدة أخرى.

الإجابة الصحيحة: B



12. لدينا كومة قصوى (Max-Heap) ممثلة بمصفوفة. إذا قمنا فجأة بتغيير قيمة عنصر يقع في موقع ورقة (Leaf) وزيادة قيمته بشكل ضخم لدرجة أنه أصبح أكبر من عقدة الجذر الحالية. ما هي العملية اللازمة للإصلاح الكومة وما تعقيدها الزمني؟

- استدعاء دالة Max_Heapify مباشرة على هذا العنصر وتكلفتها $O(N)$.
- استدعاء دالة إعادة البناء بالكامل للكومة Build_Max_Heap بتكلفة زمنية قدرها $O(N \log N)$.
- القيام بعملية "صعود للأعلى" (Shift up / Bubble up) بمقارنة العنصر المحدث مع أبيه وجده وتبديله معهما صعوداً حتى يستقر في موقعه الصحيح، وتكلفتها $O(\log N)$.
- لا يحتاج الأمر لأي عملية تعديل لأن الأوراق دائماً قيمتها معزولة وغير مؤثرة في شرط الكومة.

الإجابة الصحيحة: C

13. ما هي الميزة والمشكلة التخزينية المباشرة التي جعلت Heaps تتفوق في التمثيل عبر المصفوفات على أشجار البحث الثنائية (BST) العادية كما ورد في مطلع المحاضرة؟

- عدم إمكانية تتبع مسارات الآباء والأبناء للأشجار العادية عند تحويلها لكتل برمجية في لغة السي بلس بلس أو الجافا.
- البطء الكبير في تتبع مسارات القفزات الثنائية عند الانتقال للمصفوفات المسطحة مقارنة بالمؤشرات.
- هدر الذاكرة والشقوق الشاغرة الكبيرة في المصفوفات للشجر الثنائي غير المتوازن الذي لا يحقق صفة الاكتمال النظيف (Complete Tree)، بينما تضمن الكومة خلو المصفوفة تماماً من أي فراغات وهدر لتراكم عقدها المتجانس.
- أن الكومة تشجع على نمو الشجرة للجانب الأيمن فقط مما يوفر مسارات حرة للوصول السريع للأوراق.

الإجابة الصحيحة: C

14. كيف تسلك خوارزمية بناء الكومة القصوى Build_Max_Heap مع مصفوفة تحتوي بالكامل على بيانات مكررة متطابقة مثل [9, 9, 9, 9, 9, 9, 9] وما هو نطاق عمليات المبادلة (Swaps) التي ستجريها؟

- الوقوع في استدعاء عودي لانهائي وامتلاء مكدس الذاكرة بسبب تساوي المؤشرات عند الاختبارات.
- تعمل الخوارزمية بشكل سليم جداً وتنتج كومة صالحة، ويكون عدد عمليات المبادلة (Swaps) المنفذة هو صفر لعدم وجود اختلالات حقيقية.
- تقوم الخوارزمية بعمليات تبديل متكررة متناوبة بتكلفة زمنية $O(N^2)$ للتأكيد العشوائي على العقد ومواقعها.
- تقوم الخوارزمية بحذف الجذور تلو المكررات لتجنب الحواف الطفيلية مما يخرب مساحة التمثيل المعتمد.

الإجابة الصحيحة: B

15. ما الذي يجب أن نكتبه في [السطر 6] للتحقق من أن الابن الأيسر أكبر من العقدة الحالية؟

A. $l < n \ \&\& \ arr[l] > arr[largest]$

B. $l \leq n \ \&\& \ arr[l] > arr[i]$

C. $l < n \ \&\& \ arr[l] < arr[largest]$

D. $arr[l] > arr[largest]$

الإجابة الصحيحة: A

16. ما الذي يجب أن نكتبه في [السطر 10] للتحقق من أن الابن الأيمن أكبر من أكبر قيمة تم إيجادها حتى الآن؟

A. $r < n \ \&\& \ arr[r] > arr[i]$

B. $r < n \ \&\& \ arr[r] > arr[largest]$

C. $r \leq n \ \&\& \ arr[r] > arr[largest]$

D. $arr[r] > arr[largest]$

الإجابة الصحيحة: B

17. ما هو الاستدعاء العودي الصحيح الذي يجب كتابته في [السطر 17] لإكمال عملية الـ Heapify للشرش المتأثر؟

A. $\text{maxHeapify}(arr, n, i);$

B. $\text{maxHeapify}(arr, n, l);$

C. $\text{maxHeapify}(arr, n, largest);$

D. $\text{maxHeapify}(arr, largest, n)$

الإجابة الصحيحة: C

18. (مستوى صعب) إذا قمنا بتعديل الدالة maxHeapify لتصبح دالة تكرارية (Iterative) بدلاً من عودية (Recursive)، فما التعديل الذي يجب كتابته بدلاً من [السطر 17] وضمان استمرار الحلقة؟

A. $i = largest;$

B. $i = l;$

C. $\text{return};$

D. $i = 2 * i + 1$

الإجابة الصحيحة: C

19. ما هو الشرط الصحيح الذي يجب وضعه داخل حلقة while في [السطر 7] عند إدخال عنصر جديد لضمان الحفاظ على خصائص الـ Max-Heap صعوداً؟

A. $arr[i] < arr[(i - 1) / 2]$

B. $arr[i] > arr[i / 2]$

C. $arr[i] > arr[(i - 1) / 2]$

D. $arr[i] != arr[(i - 1) / 2]$

الإجابة الصحيحة: C

20. ما الذي يجب كتابته في [السطر 11] لتحديث مؤشر العقدة الحالية والانتقال للعقدة الأب (Parent) داخل الحلقة؟

A. $i = i / 2;$

B. $i = (i - 1) / 2;$

C. $i = 2 * i + 1;$

D. $i = i - 1$

الإجابة الصحيحة: B

21. (مستوى صعب) في تابع بناء الهيب buildHeap، ما هي القيمة الابتدائية لـ i في [السطر 3] لتبدأ الحلقة من آخر عقدة غير ورقية (Last non-leaf node) لتقليل التعقيد الزمني إلى $O(n)$ ؟

A. $n / 2$

B. $(n / 2) - 1$

C. $n - 1$

D. $(n - 2) / 2$

الإجابة الصحيحة: B

DONE ☒

عاش النوحش والبطولة 🤖😄

نلتقكم في الملتح القادم